

BuildStream: A Reproducible AWS-Native Reference Architecture for Mobile CI/CD with Ephemeral Code-Signing Credentials

Santosh Kumar Puppala
Independent Researcher
Pennsylvania, USA
puppalas97@gmail.com

Abstract—Mobile application delivery imposes security and operational burdens that web deployment does not: iOS builds require macOS hardware, code signing depends on long-lived secret material (Apple .p12 certificates and Android keystores), and distribution spans multiple gated app stores. Common practice either stores signing credentials on persistent build machines or entrusts them to third-party hosted continuous-integration (CI) services, both of which widen the exposure surface for the most sensitive assets in the mobile software supply chain. We present *BuildStream*, an open-source, AWS-native reference architecture for mobile CI/CD that keeps signing credentials inside the organization’s own cloud account and reduces their on-disk lifetime to the duration of a single signing operation. Its central mechanism is a *pull-use-destroy* ephemeral-credential pattern: signing material is retrieved from AWS Secrets Manager at sign time, used to sign exactly one artifact, and erased—together with any transient keychain—before the workspace is purged. A Jenkins shared-library design routes iOS and Android builds through a common pipeline that an application adopts with a three-line configuration. We describe the threat model, the architecture across its networking, orchestration, build, secrets, artifact, and monitoring layers, and the signing flows for both platforms. We report a working deployment on Amazon EC2—including an EC2 Mac Dedicated Host—with end-to-end iOS and Android pipeline runs, versioned artifacts in S3, and CloudWatch monitoring, and we analyze the resulting credential-exposure window. We discuss portability to Bitbucket and IBM UrbanCode Deploy, and we are explicit about the limitations of the present prototype.

Index Terms—mobile CI/CD, code signing, secret management, DevSecOps, cloud infrastructure, ephemeral credentials, least privilege, AWS.

I. INTRODUCTION

Continuous integration and continuous delivery (CI/CD) for mobile applications is materially harder than for web services. Building an iOS application requires Apple’s toolchain on macOS hardware; building Android requires a specific SDK and Gradle toolchain; and *both* platforms gate release artifacts behind mandatory code signing. Signing in turn depends on long-lived secret material: an Apple distribution certificate exported as a PKCS#12 (.p12) bundle together with a provisioning profile for iOS, and a Java KeyStore (.jks) for Android. These keys are among the most sensitive assets an

organization holds. On Android in particular, the signing key is bound permanently to an application’s package identity: losing or rotating it severs the ability to ship updates to the existing install base [1].

Two operational patterns dominate in practice, and each enlarges the attack surface around exactly these assets. In the first, an organization runs persistent build servers and stores certificates and keystores on disk so that builds can find them. The credential then lives on the machine indefinitely, readable by any process that compromises the build environment, and its blast radius spans every build the server ever runs. In the second, the organization adopts a third-party hosted CI service and uploads its signing credentials to that vendor. Convenience improves, but the most sensitive secrets in the mobile supply chain now leave the organization’s own trust boundary, and regulated sectors (finance, healthcare, government) frequently cannot accept that custody transfer at all.

Position. We argue that mobile signing credentials should (i) never leave the organization’s own cloud account and (ii) exist on a build agent only for the few seconds in which a signature is actually produced. BuildStream is a concrete, open-source instantiation of that position on Amazon Web Services (AWS).

Contributions. This paper makes the following contributions:

- 1) A *pull-use-destroy* ephemeral-credential pattern for mobile code signing, instantiated for both Apple .p12 signing (via a temporary, immediately-destroyed keychain) and Android keystore signing (via a transient decoded .jks), with build-log hygiene to prevent secret leakage (Section V).
- 2) A complete AWS-native reference architecture that keeps credentials in-account, isolates build agents in a private network, and enforces least-privilege IAM scoping per secret namespace (Section IV).
- 3) A Jenkins shared-library implementation that provides zero-configuration, three-line onboarding for any mobile application while centralizing all pipeline logic at a single point of maintenance (Section VI).

- 4) An open-source reference implementation and a working-deployment evaluation on real AWS infrastructure, including measured credential-exposure windows for both platforms (Section VII).

We are deliberately explicit about scope. BuildStream is a *reference architecture and prototype*, not a peer-reviewed production study. Our evaluation demonstrates architectural viability and a quantifiable reduction in credential exposure; it is not a statistical performance benchmark and does not claim production adoption. Section VIII-C states precisely what is and is not demonstrated. All code and documentation are publicly available.¹

II. BACKGROUND AND RELATED WORK

A. Mobile Code Signing

Apple requires every distributed iOS binary to be signed with a certificate issued by an enrolled developer account and paired with a provisioning profile that authorizes specific devices or distribution channels [2]. The private key and certificate are typically managed as a .p12 bundle and imported into a macOS keychain so that `xcodebuild` can sign during export. Android requires that every APK or App Bundle (AAB) be signed with a key held in a Java KeyStore; the platform pins an application’s package name to its signing key for the lifetime of the app [1]. Google Play App Signing optionally lets Google hold the distribution key while the developer retains an “upload key,” but the developer-side signing step is unchanged. These mechanisms make the signing key a long-lived, high-value, and—on Android—effectively irreplaceable secret.

B. Secret Management and Just-in-Time Credentials

The principle of least privilege [3] argues that a component should hold only the authority it needs, for only as long as it needs it. Modern secret managers operationalize this for credentials: AWS Secrets Manager [4] and HashiCorp Vault [5] centralize secret storage behind access policies and audit logging, and Vault popularized *dynamic* or short-lived secrets that are issued on demand and expire. National guidance on secure software supply chains, including the NIST Secure Software Development Framework [6], similarly emphasizes protecting and minimizing exposure of signing material. BuildStream applies the just-in-time idea to a domain where the secret itself cannot be made short-lived—an Android signing key must persist for years—by instead minimizing the *on-agent lifetime* of a copy of that secret.

C. CI/CD for Mobile

Jenkins [7] is a widely deployed self-hosted automation server whose shared-library mechanism allows pipeline logic to be centralized and reused across repositories. Fastlane [8] provides higher-level actions for iOS and Android build, signing, and store submission. Hosted services such as Bitrise, CircleCI, and GitHub Actions [9] reduce operational effort

but require the organization to store signing credentials within the vendor’s platform and offer limited control over the build environment—particularly for macOS, where access to dedicated Apple hardware is constrained. AWS offers EC2 Mac Dedicated Hosts that satisfy Apple’s macOS licensing terms and allow iOS builds to run on instances inside the customer’s own account and network [10]. BuildStream combines self-hosted Jenkins, Fastlane, and EC2 Mac with AWS-native secret and artifact services so that the entire signing path—hardware, credentials, and artifacts—remains within a single AWS account boundary.

III. THREAT MODEL AND DESIGN GOALS

Assets. The primary protected assets are the signing credentials (Apple .p12 certificate and password, provisioning profile, Android keystore and its passwords). Secondary assets are application source code and build artifacts.

Adversaries and assumptions. We assume the AWS control plane and the Jenkins controller are trusted. We treat build agents as *semi-trusted*: an agent is correctly provisioned but a build job could be subverted at runtime—for example by a malicious dependency or a compromised build script—and could attempt to read whatever credential material is present on disk or in the build log. We also consider a network adversary attempting to reach agents directly, and a low-privilege insider holding scoped IAM credentials. We do not defend against a full compromise of the AWS account root or of the Jenkins controller host.

Design goals. From this model we derive:

- G1** *No persistence*: signing credentials never remain on a build agent between builds.
- G2** *In-account custody*: credentials never leave the organization’s AWS account.
- G3** *Least privilege*: an agent can read only the secret namespace and artifact bucket it requires.
- G4** *Network isolation*: build agents accept no inbound traffic except from the controller.
- G5** *Log hygiene*: secret values never appear in build logs.
- G6** *Low onboarding cost*: adopting the pipeline requires minimal per-application configuration.
- G7** *Auditability*: build and access events are centrally logged.

IV. SYSTEM ARCHITECTURE

BuildStream is organized into six layers, summarized in Table I and depicted in Fig. 1.

A. Network Isolation (G4)

The deployment provisions a Virtual Private Cloud (VPC) with a public subnet for the Jenkins controller and a private subnet for the Mac build agents. The agents’ security group permits inbound SSH only from the controller’s security group; they expose no public interface. The controller, reachable by administrators on its web port, dispatches jobs to agents over SSH.

¹<https://github.com/Santoshkumarpuppala/BuildStream>

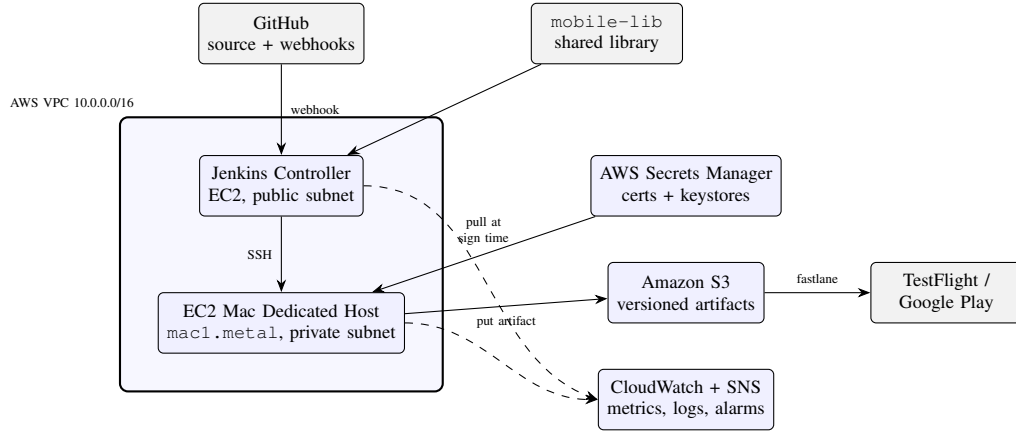


Fig. 1. BuildStream reference architecture. Build agents run in a private subnet that accepts inbound connections only from the Jenkins controller. Signing credentials are pulled from Secrets Manager directly onto the agent at sign time and never traverse the controller. Signed artifacts are written to a versioned S3 bucket; distribution to TestFlight or Google Play occurs only on the main branch.

TABLE I
ARCHITECTURAL LAYERS AND THE GOALS THEY ADDRESS.

Layer	Components	Goals
Source control	GitHub repositories, webhooks	G6, G7
Orchestration	Jenkins controller (EC2, public subnet); mobile-lib shared library	G6
Build	EC2 Mac Dedicated Host (private subnet); Xcode, Gradle, Fastlane	G1, G4
Secrets	AWS Secrets Manager; scoped IAM roles	G2, G3
Artifacts	Amazon S3 (versioned bucket)	G3, G7
Monitoring	CloudWatch agent, alarms, SNS	G7

B. Least-Privilege IAM (G3)

The controller and the agents assume distinct IAM roles. The policy attached to build infrastructure scopes Secrets Manager access to two namespaces, `ios/*` and `android/*`, and scopes S3 access to a single artifact bucket. An agent therefore cannot read secrets outside the mobile-signing namespaces, nor write to any other bucket. Listing 1 shows the core of the policy.

```
{
  "Effect": "Allow",
  "Action": ["secretsmanager:GetSecretValue"],
  "Resource": [
    "arn:aws:secretsmanager:us-east-1::secret:ios/*",
    "arn:aws:secretsmanager:us-east-1::secret:android/*" ]
},
{
  "Effect": "Allow",
  "Action": ["s3:PutObject", "s3:GetObject", "s3:ListBucket"],
  "Resource": [
    "arn:aws:s3::myhub-mobile-artifacts",
    "arn:aws:s3::myhub-mobile-artifacts/*" ] }
```

Listing 1. Least-privilege policy fragment scoping secret and artifact access.

C. Versioned Artifact Storage (G7)

Signed artifacts are written to a versioned S3 bucket under a structured key that encodes platform, application, branch, and build number:

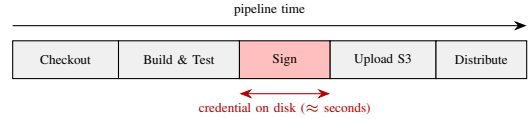


Fig. 2. Credential-exposure window. A decoded signing key exists on the agent only during the Sign stage; it is erased before Upload, and the workspace is purged after Distribute.

`<platform>/<app>/<branch>/<build>/<app>.<ext>`

This scheme makes every build independently addressable, supports per-branch retention via S3 lifecycle rules, and provides an audit trail linking each artifact to the commit and build that produced it.

V. THE EPHEMERAL CREDENTIAL PATTERN

The core security mechanism of BuildStream is a *pull–use–destroy* discipline applied uniformly to signing material:

- 1) **Pull.** At the start of the signing stage, the agent retrieves the required secrets from Secrets Manager using its scoped IAM role.
- 2) **Use.** The decoded material is used to sign exactly one artifact.
- 3) **Destroy.** Every transient artifact—the decoded key file and any keychain created to hold it—is deleted immediately after signing, and the entire workspace is purged at the end of the build.

Figure 2 situates the resulting exposure window within the pipeline: a decoded copy of the signing key exists on the agent only during the *Sign* stage, which in our measurements lasts a few seconds (Section VII), in contrast to the indefinite on-disk lifetime of the persistent-server baseline.

A. Android Signing

The Android signing routine pulls the keystore and its passwords through a small helper that extracts named fields from

each JSON secret, decodes the keystore to a temporary .jks, invokes jarsigner, and then removes the keystore. Shell tracing is disabled (set +x) so that passwords never reach the build log (G5). Listing 2 reproduces the implementation.

```
def sign(String appName) {
    def aws = new AwsSecrets(script)
    def ksB64 = aws.getSecretField('android/keystore','keystore')
    def ksPwd = aws.getSecretField('android/keystore-password','password')
    def alias = aws.getSecretField('android/key-alias','alias')
    def keyPwd = aws.getSecretField('android/key-password','password')
    script.sh """
        set +x # suppress trace: no passwords in build log
        echo '${ksB64}' | base64 -d > keystore.jks
        AAB=$(find . -name "*.aab" | head -1)
        jarsigner -sigalg SHA256withRSA -digestalg SHA-256 \
            -keystore keystore.jks -storepass '${ksPwd}' \
            -keypass '${keyPwd}' "${AAB}" '${alias}'
        mkdir -p output && cp "${AAB}" output/${appName}.aab
        rm -f keystore.jks # destroy ephemeral credential
    """
}
```

Listing 2. Android pull-use-destroy signing (Groovy).

B. iOS Signing

The iOS production signing path follows the same discipline using a *temporary keychain*. The .p12 certificate and its password are pulled and decoded; a dedicated keychain is created with security create-keychain; the certificate is imported; the archive is exported and signed with xcodebuild -exportArchive; and the keychain is then destroyed with security delete-keychain, followed by removal of the decoded certificate file. The keychain exists only for the duration of the export and is never added to the agent’s login keychain, so no signing identity persists.

C. Defense-in-Depth Summary

The pattern jointly satisfies G1, G2, and G5: credentials are fetched in-account under a scoped role (G2/G3), exist on disk only transiently and are explicitly destroyed (G1), and are kept out of logs by disabling shell tracing around their use (G5). The end-of-build workspace purge (cleanWs) provides a belt-and-suspenders guarantee that nothing survives the build.

VI. IMPLEMENTATION

BuildStream’s pipeline logic is packaged as a Jenkins shared library, mobile-lib, comprising a global pipeline function and a small set of per-concern classes (an AWS secrets helper, an iOS pipeline, an Android pipeline, and a notifier). Centralizing the logic means an improvement deploys to every application at once and no application repository contains pipeline code beyond a three-line declaration (Listing 3), satisfying G6.

```
@Library('mobile-lib') _
mobilePipeline(platform: 'ios', appName: 'MyHub')
```

Listing 3. Complete CI/CD integration for an application.

A. Platform Routing

A single entry point, mobilePipeline, derives the triggering branch, validates the requested platform, and dispatches each stage to the appropriate platform class. Distribution is gated on the main branch so that feature branches build and test without signing or publishing. Listing 4 shows the routing skeleton.

```
def call(Map config = [:]) {
    def platform = config.platform?.toLowerCase()
    def appName = config.appName ?: 'UnknownApp'
    def branch = env.BRANCH_NAME ?: 'main'
    def isMain = (branch == 'main')
    if (!platform) error "platform required: 'ios' or 'android'"
    pipeline {
        agent { label 'mac-agent' }
        options { disableConcurrentBuilds(); timeout(time:60, unit:'MINUTES') }
        stages {
            stage('Build & Test') { steps { script {
                if (platform=='ios') new IosPipeline(this).
                    buildAndTest(appName)
                else new AndroidPipeline(this).
                    buildAndTest(appName)
            }}}
            stage('Sign') { /* platform sign() */ }
            stage('Upload to S3') { /* structured key */ }
            stage('Distribute') { when { expression { isMain } } }
        }
        post { always { script { /* notify */ }; cleanWs() } }
    }
}
```

Listing 4. Platform routing and branch gating (abridged).

B. Operational Concerns

The pipeline disables concurrent builds to guarantee workspace isolation, sets a build timeout, and rotates build history. A notifier reports the final result (and is wired for Slack delivery). The same library serves both reference applications used in our evaluation, myhub-ios and myhub-android, unchanged.

VII. EVALUATION

We deployed BuildStream on a real AWS account in us-east-1: a Jenkins controller on EC2, an EC2 Mac Dedicated Host registered as the build agent (mac-mini-agent), Secrets Manager entries for the iOS and Android signing material, a versioned S3 bucket, and the CloudWatch agent on the build host. We then exercised the pipeline end to end for both platforms. Our evaluation answers three questions: (Q1) does the architecture function end to end on real infrastructure? (Q2) what credential-exposure window does the ephemeral pattern actually yield? (Q3) what is the onboarding cost?

A. Q1: End-to-End Functionality

A library-load job confirmed that mobile-lib loads and exposes the pipeline entry point. A verification job confirmed that the agent has the required toolchain and—using only its scoped IAM role—can enumerate the expected secrets (ios/apple-pl2-cert, ios/apple-pl2-password, android/keystore); see Fig. 3. Full pipeline runs then completed all stages for both platforms (Fig. 4), producing signed artifacts at the expected versioned S3 keys

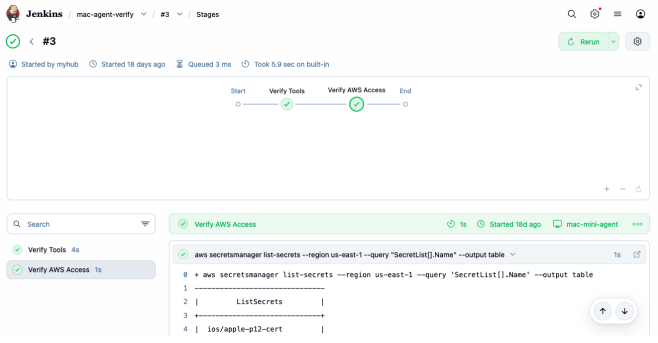


Fig. 3. Agent-side verification: using only its scoped instance role, the build agent enumerates the mobile-signing secrets via the Secrets Manager API.

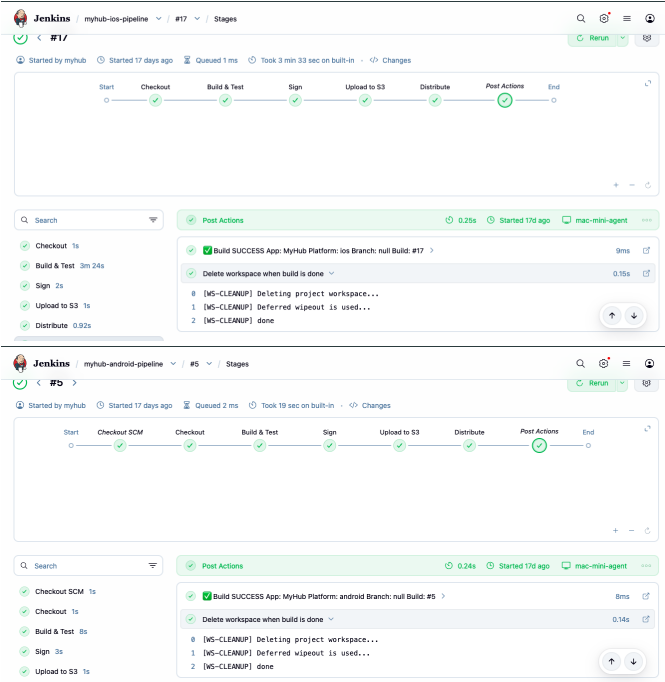


Fig. 4. End-to-end pipeline runs. Top: iOS build #17 (total 3 m 33 s). Bottom: Android build #5 (total 19 s). Both complete Checkout, Build & Test, Sign, and Upload to S3; Distribute is gated on main.

(Fig. 5): ios/MyHub/main/17/MyHub.ipa (5.4 MB) and android/MyHub/main/5/MyHub.aab (20.7 MB).

B. Q2: Credential-Exposure Window

Table II reports per-stage durations for both runs. The decoded signing key exists on disk only for the Sign stage: at most 2 s for the iOS path and 3 s for the Android path in these runs. The workspace is then purged. We contrast this with the persistent-server baseline, in which a copy of the credential resides on disk continuously. The reduction is qualitative as much as quantitative: exposure shifts from “always present, broad blast radius” to “present for seconds, scoped to one signing operation.”

The order-of-magnitude difference in Build & Test time between platforms (xcodebuild compiling and running

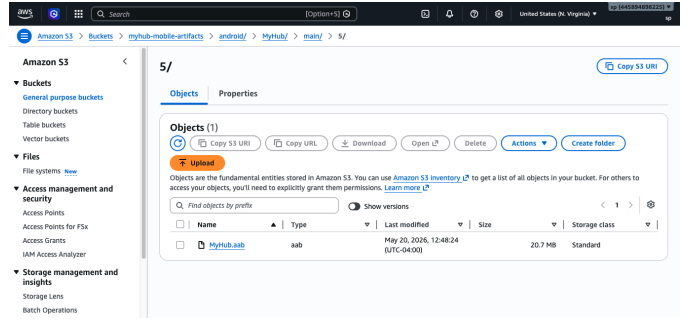


Fig. 5. A signed artifact at its versioned key, android/MyHub/main/5/, linking the AAB to the exact build that produced it.

TABLE II
PER-STAGE DURATIONS (SINGLE RUN PER PLATFORM).

Stage	iOS #17	Android #5
Checkout	1 s	1 s
Build & Test	3 m 24 s	8 s
Sign (<i>exposure window</i>)	2 s	3 s
Upload to S3	1 s	1 s
Distribute	0.92 s	—
Post (notify + purge)	0.25 s	0.24 s
Total	3 m 33 s	19 s

tests on an iOS simulator versus an incremental Gradle build with a warm cache) is incidental to the security mechanism; the exposure window is governed by the Sign stage alone and is small on both platforms.

C. Q3: Onboarding Cost and Monitoring

Adopting the full pipeline requires the three-line declaration of Listing 3 and no other per-application CI code. On the operability side, the CloudWatch agent forwards three log groups (/jenkins/builds, /jenkins/system, /mac-agents/jenkins-agent) and a CPU alarm on the build host remained in the OK state with the agent idle at roughly 4–5% CPU between builds, confirming that metrics flow from the private-subnet host to CloudWatch (G7).

VIII. DISCUSSION

A. Portability

Because all pipeline logic resides in the shared library and the only source-control coupling is a repository URL and a credential identifier, migrating from GitHub to Bitbucket requires no change to pipeline code—only new credentials and webhook configuration. Similarly, an enterprise release gate such as IBM UrbanCode Deploy can be inserted as an additional publish stage without disturbing the build or signing stages. Both integrations are documented in the repository.

B. Scalability

Agents share a Jenkins label and are load-balanced automatically; throughput scales horizontally by registering additional EC2 Mac hosts under the same label. We did not load-test this dimension.

C. Limitations and Threats to Validity

We state these plainly, as they bound our claims:

- **iOS signing is demonstrated by design, not by execution with a production certificate.** In our prototype the iOS Sign stage packages a simulator build (`CODE_SIGNING_ALLOWED=NO`); the production temporary-keychain signing path is implemented as the documented `security/xcodebuild` sequence but was not exercised against a paid Apple Developer distribution certificate. The Android path *is* fully exercised end to end with a real keystore and `jarsigner`.
- **Distribution is stubbed.** The Distribute stages contain the production Fastlane invocations but no live store upload was performed, as that requires paid App Store and Google Play accounts.
- **Measurements are illustrative, not statistical.** Timings are single runs of one application on one warm agent; they are not a benchmark and we make no head-to-head performance comparison against hosted CI.
- **Single-agent deployment.** Horizontal scaling and controller high-availability are designed but unmeasured.
- **No third-party audit or external adoption.** The security properties are argued from the design and the observed exposure window; they have not been independently audited.

Within these bounds, the contribution is the architecture and the ephemeral-credential pattern, demonstrated end to end for Android and for the non-signing portions of the iOS pipeline, with the iOS signing path specified in full.

IX. CONCLUSION

BuildStream is an open-source reference architecture that brings mobile signing credentials back inside an organization’s own AWS account and shrinks their on-agent lifetime from indefinite to seconds. Its pull–use–destroy pattern, least-privilege IAM scoping, private-subnet build agents, and Jenkins shared-library design together provide a secure yet low-friction path to mobile CI/CD: applications onboard in three lines, and signing credentials are present on a build host only for the moment a signature is produced. We reported a working AWS deployment with end-to-end pipeline runs, versioned artifacts, and monitoring, and we quantified the resulting exposure window while being explicit about the prototype’s limits. The complete implementation and documentation are publicly available to serve as a reproducible starting point for organizations that cannot, or will not, surrender custody of their signing keys.

REFERENCES

- [1] Android Developers, “Sign your app,” Google. <https://developer.android.com/studio/publish/app-signing>.
- [2] Apple Inc., “Code Signing,” Apple Developer Documentation. <https://developer.apple.com/documentation/security/code-signing-services>.
- [3] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [4] Amazon Web Services, “AWS Secrets Manager User Guide,” 2024. <https://docs.aws.amazon.com/secretsmanager/>.
- [5] HashiCorp, “Vault: Secrets management,” 2024. <https://www.vaultproject.io/>.
- [6] M. Souppaya, K. Scarfone, and D. Dodson, “Secure Software Development Framework (SSDF) Version 1.1,” NIST SP 800-218, 2022.
- [7] Jenkins Project, “Jenkins User Documentation: Shared Libraries,” 2024. <https://www.jenkins.io/doc/book/pipeline/shared-libraries/>.
- [8] Fastlane, “Fastlane docs,” 2024. <https://docs.fastlane.tools/>.
- [9] GitHub, “GitHub Actions Documentation,” 2024. <https://docs.github.com/actions>.
- [10] Amazon Web Services, “Amazon EC2 Mac Instances,” 2024. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-mac-instances.html>.